Projeto de Deep Learning

Etapa 1: Estudo e Comparação de Embeddings

Nesta etapa, cada grupo deverá escolher um domínio de aplicação, estudar o conceito de **embeddings**, gerar representações com uma abordagem de **deep learning** e com uma abordagem **tradicional**, comparar os resultados e, ao final, utilizar o espaço vetorial aprendido como entrada para um **método não baseado em deep learning**. A ideia central é compreender como diferentes formas de representação influenciam a qualidade da solução construída.

1. Geração de embeddings com deep learning

Nesta etapa, o objetivo é gerar uma representação vetorial densa dos dados utilizando uma abordagem baseada em redes neurais. Em outras palavras, vocês devem transformar cada amostra do conjunto de dados em um vetor numérico que capture características relevantes do problema.

Em geral, há dois caminhos principais:

- **Usar um modelo pré-treinado** e aproveitar suas camadas intermediárias como extrator de características.
- **Treinar um modelo** e usar a saída de uma camada intermediária ou penúltima como embedding.

> A ideia mais importante aqui é a seguinte: em uma rede de classificação, a camada final normalmente serve apenas para produzir a decisão final do modelo. Já as camadas anteriores costumam conter uma representação mais rica e mais geral dos dados. É justamente essa representação intermediária que normalmente utilizamos como **embedding**.

1.1 Estratégia recomendada

1. Escolher um dataset e definir o que será representado.
2. Escolher uma arquitetura adequada ao domínio.
3. Carregar um modelo pré-treinado ou treinar um modelo simples.
4. Remover a camada final, ou ignorá-la, e utilizar a saída de uma camada anterior como embedding.
5. Gerar um vetor para cada amostra do dataset.
6. Salvar esses vetores para comparação com métodos tradicionais e uso na aplicação final.

1.2 Como escolher a camada de embedding

Nem toda camada é uma boa escolha. Em geral:

- camadas muito iniciais capturam padrões simples, como bordas, texturas ou estruturas locais;
- camadas intermediárias combinam padrões mais relevantes;
- camadas muito finais tendem a ficar mais especializadas na tarefa de classificação original.

Por isso, uma escolha comum é usar:

- a **penúltima camada** em redes densas;
- a saída de um **global average pooling** em CNNs;
- uma camada intermediária especifica, quando o grupo quiser comparar diferentes níveis de abstração.

1.3 Exemplo prático: extração de features em Keras / TensorFlow

Um caminho bastante comum em imagens é usar uma rede pré-treinada sem a camada final de classificação:

```
from tensorflow
import keras
base_model = keras.applications.ResNet50(weights="imagenet", include_top=False, pooling="avg", input_shape=
(224, 224, 3) )
base_model.trainable = False
embeddings = base_model.predict(x_images)
```

Nesse exemplo:

- `include_top=False` remove a parte final da rede, responsável pela classificação original;
- `pooling="avg"` transforma o mapa de ativação final em um vetor;
- o resultado de `.predict()` já é o embedding de cada imagem.

1.4 Exemplo prático: extração de features em PyTorch

Em PyTorch, uma solução simples é substituir a camada final por uma identidade.

```
import torch
import torch.nn as nn
from torchvision import models
model = models.resnet50(weights="DEFAULT")
model.fc = nn.Identity()
model.eval()
with torch.no_grad():
    embeddings = model(x_images)
```

Nesse caso, a rede continua executando todas as camadas anteriores, mas a última camada classificadora deixa de ser usada. O vetor retornado passa a ser a representação de cada amostra.

1.5 O que o grupo deve explicar no trabalho

- qual arquitetura foi escolhida;
- qual camada foi usada como embedding;
- qual a dimensão do vetor gerado;
- se o modelo foi pré-treinado ou treinado do zero;
- quais vantagens e limitações essa escolha trouxe.

```
Exemplos de domínios e modelos
Texto: Word2Vec, FastText, autoencoders, modelos baseados em transformers
Imagens: ResNet, EfficientNet, MobileNet, autoencoders convolucionais
Audio: CNNs sobre espectrogramas, autoencoders, redes recorrentes
```

2. Geração de embeddings com métodos tradicionais

Aqui, a proposta é construir uma representação alternativa **sem usar deep learning**. Esses métodos podem servir como baseline e também ajudam a entender melhor o que a abordagem profunda realmente está aprendendo.

Diferentemente das redes neurais, os métodos tradicionais normalmente se apoiam em:

- estatísticas sobre os dados;
- transformações lineares;
- fatoração de matrizes;
- descritores projetados manualmente para capturar padrões relevantes.

2.1 Exemplos por domínio

Texto

- **TF-IDF**: representa o documento pela importância relativa de cada termo.
- **Bag of Words**: representa pela frequência bruta de palavras.
- **LSA / TruncatedSVD**: reduz a dimensionalidade da matriz termo-documento e gera um espaço latente mais compacto.

Imagem

- **SIFT**: descritores locais construídos em torno de pontos de interesse.
- **SURF**: proposta semelhante ao SIFT, com foco em eficiência.
- **ORB**: alternativa mais leve para detectar e descrever pontos locais.
- **HOG**: captura distribuição de gradientes e é muito usado em forma e textura.
- **LBP**: descreve padrões locais de textura.
- **GIST**: produz um descritor global da cena.

Áudio

- **MFCC**: muito usado para representar características espectrais do sinal.
- estatísticas de energia, frequência e espectro;
- agregações temporais sobre janelas do sinal.

2.2 Onde encontrar esses descritores

Descritor / Método	Biblioteca	Observação
TF-IDF / Bag of Words	scikit-learn	Bom para texto, simples de aplicar e interpretar.
PCA / TruncatedSVD	scikit-learn	Úteis para redução de dimensionalidade e projeção em espaço latente.
SIFT / SURF	OpenCV	Descritores clássicos locais para imagens.
ORB	OpenCV ou scikit-image	Alternativa leve, útil em tarefas com pontos de interesse.
HOG	scikit-image	Muito usado para forma e contorno.
LBP	scikit-image	Útil para textura.
MFCC	librosa	Descritor tradicional bastante usado em áudio.

2.3 Exemplos práticos

Exemplo para texto: TF-IDF + SVD

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
vectorizer = TfidfVectorizer(max_features=5000)
X_tfidf = vectorizer.fit_transform(textos)
svd = TruncatedSVD(n_components=128, random_state=42)
embeddings_trad = svd.fit_transform(X_tfidf)
```

Exemplo para imagem: HOG

```
from skimage.feature import hog
from skimage.color import rgb2gray
img_gray = rgb2gray(img)
features = hog(img_gray, pixels_per_cell=(8, 8), cells_per_block=(2, 2))
```

Exemplo para imagem: ORB

```
from skimage.feature import ORB
from skimage.color import rgb2gray
img_gray = rgb2gray(img)
extractor = ORB(n_keypoints=200)
extractor.detect_and_extract(img_gray)
descriptors = extractor.descriptors
```

Atenção: alguns descritores locais, como SIFT, SURF e ORB, produzem um conjunto variável de descritores por imagem. Nesses casos, o grupo precisa pensar em como transformar isso em um vetor fixo por amostra. Algumas opções são:

- usar média ou soma dos descritores;
- usar Bag of Visual Words;
- usar histogramas ou estatísticas agregadas.

> O mais importante aqui não é apenas aplicar um descritor, mas justificar por que ele faz sentido para o domínio escolhido e como ele será comparado com o embedding obtido por deep learning.

3. Avaliação e visualização dos embeddings

Depois de gerar os embeddings, é preciso analisá-los. Uma forma muito comum de fazer isso é visualizar os vetores em duas dimensões e observar se amostras parecidas ficam próximas entre si, enquanto amostras de grupos diferentes aparecem mais separadas.

Como embeddings costumam ter muitas dimensões, usamos métodos de **redução de dimensionalidade** para projetar esses vetores em 2D ou 3D.

3.1 PCA

O **PCA** é um método linear de redução de dimensionalidade. Ele procura novas direções no espaço dos dados que expliquem a maior parte possível da variância observada. Em termos intuitivos, ele tenta encontrar os eixos mais informativos para representar os dados com menos dimensões.

O PCA é útil porque:

- é rápido e simples de interpretar;
- serve como primeira análise exploratória;
- pode reduzir ruído e redundância;
- também pode ser usado antes de outros métodos, como t-SNE.

Limitação: como ele é linear, pode não capturar estruturas complexas e curvas no espaço dos embeddings.

3.2 t-SNE

O **t-SNE** é um método não linear muito usado para visualização. Ele tenta preservar relações de vizinhança local: pontos que eram próximos no espaço original tendem a continuar próximos no espaço projetado.

Intuitivamente, ele é bom para revelar grupos, manchas e estruturas locais escondidas em espaços de alta dimensão.

O t-SNE é interessante porque:

- gera visualizações frequentemente muito boas para explorar clusters;
- é muito popular em análise de embeddings;
- ajuda a observar coesão local dos grupos.

Limitações:

- é mais lento;
- sensível a parâmetros;
- as distâncias globais do gráfico nem sempre devem ser interpretadas literalmente.

3.3 UMAP

O **UMAP** também é um método não linear de redução de dimensionalidade. Assim como o t-SNE, ele busca preservar a estrutura dos dados, mas costuma equilibrar melhor relações locais e parte da estrutura global.

Em muitos casos, o UMAP:

- é mais rápido que o t-SNE;
- preserva melhor a organização geral dos dados;
- funciona muito bem para visualização e exploração de embeddings.

Na prática, PCA, t-SNE e UMAP não competem exatamente entre si como se apenas um fosse o correto. Muitas vezes, a melhor estratégia é usar mais de um deles e comparar o que cada visualização revela.

3.4 Exemplo de uso

PCA

```
from sklearn.decomposition import PCA
X_pca = PCA(n_components=2, random_state=42).fit_transform(embeddings)
```

t-SNE

```
from sklearn.manifold import TSNE
X_tsne = TSNE(n_components=2, random_state=42, init="pca").fit_transform(embeddings)
```

UMAP

```
import umap
X_umap = umap.UMAP(n_components=2, random_state=42).fit_transform(embeddings)
```

3.5 O que observar nas visualizações

- amostras da mesma classe tendem a ficar próximas?
- há separação visual entre grupos?
- há regiões com muita sobreposição?
- o embedding profundo é o tradicional produzem organizações diferentes?

3.6 Métricas quantitativas

A visualização ajuda muito, mas ela sozinha não basta. Sempre que possível, o grupo deve complementar a análise com métricas quantitativas, como:

- separação entre classes;
- métricas de clustering, como silhouette score;
- desempenho em classificação, recuperação ou regressão usando os embeddings como entrada.

4. Aplicação final com método não baseado em deep learning

Após gerar e comparar os embeddings, o grupo deverá utilizá-los como entrada para um método **não baseado em deep learning**. O objetivo é mostrar se a representação aprendida ajuda efetivamente a resolver um problema.

Exemplos de métodos finais

- k-NN
- SVM
- Logistic Regression
- K-Means
- Agglomerative Clustering

5. Referências e guias para complementar o projeto

Abaixo estão alguns materiais externos recomendados para aprofundar a implementação, entender melhor as bibliotecas utilizadas e encontrar exemplos práticos para cada etapa do projeto.

5.1 Deep learning e extração de embeddings

Tema	Guia / referência	Uso sugerido
Transfer learning e feature extraction	Keras - Transfer learning & fine-tuning	Para usar modelos pré-treinados e remover a camada final de classificação.
Extração de ativações intermediárias	Torchvision - Feature extraction for model inspection	Para capturar embeddings de camadas intermediárias em modelos PyTorch.

5.2 Métodos tradicionais para texto

Método	Guia / referência	Uso sugerido
TF-IDF	scikit-learn - TfidfVectorizer	Para gerar embeddings esparsos de documentos ou textos.
LSA / TruncatedSVD	scikit-learn - TruncatedSVD	Para reduzir TF-IDF a um espaço latente mais compacto.

5.3 Métodos tradicionais para imagem

Descritor	Guia / referência	Uso sugerido
ORB	scikit-image - ORB feature detector and binary descriptor	Bom para detectar e descrever pontos locais de interesse.
HOG	scikit-image - Histogram of Oriented Gradients	Útil para capturar forma, contorno e gradientes da imagem.
LBP	scikit-image - Local Binary Pattern	Interessante para análise de textura.
SIFT	OpenCV - SIFT	Descritor clássico local para pontos-chave e correspondência entre imagens.
Documentação geral	scikit-image - documentação principal	Para procurar outros descritores e exemplos adicionais.

5.4 Métodos tradicionais para áudio

Método	Guia / referência	Uso sugerido
MFCC	librosa - librosa.feature.mfcc	Para extrair coeficientes cepstrais em frequência Mel de sinais de áudio.
Biblioteca principal	librosa - documentação geral	Para explorar outras features e exemplos em áudio.

5.5 Redução de dimensionalidade e visualização

Método	Guia / referência	Uso sugerido
PCA	scikit-learn - PCA	Para redução linear de dimensionalidade e análise inicial do espaço vetorial.
t-SNE	scikit-learn - TSNE	Para visualizar vizinhanças locais e possíveis agrupamentos.
UMAP	UMAP - How to Use UMAP	Para visualização não linear e exploração da estrutura do embedding.